

USENIX

THE ADVANCED COMPUTING SYSTEMS ASSOCIATION

# Strata：面向长上下文语言模型服务的分层上下文缓存

Zhiqiang Xie, *Stanford University and NVIDIA;* Ziyi Xu, *Shanghai Jiao Tong University;* Mark Zhao, *University of Colorado Boulder;* Yuwei An, *Carnegie Mellon University;* Vikram Sharma Mailthody, *NVIDIA;* Scott Mahlke, *NVIDIA and University of Michigan;* Michael Garland, *NVIDIA;* Christos Kozyrakis, *NVIDIA and Stanford University*

https://www.usenix.org/conference/osdi26/presentation/xie-zhiqiang

本文收录于第 20 届 USENIX 操作系统设计与实现研讨会论文集。

7 月 13 日至 15 日，　　美国华盛顿州西雅图

ISBN 978-1-939133-55-7

第 20 届 USENIX 操作系统设计与实现研讨会论文集的开放获取由以下机构赞助



جامعة الملك عبدالله للعلوم والتقنية

King Abdullah University of Science and Technology

# Strata：面向长上下文语言模型服务的分层上下文缓存

Zhiqiang Xie
*Stanford University & NVIDIA*

Ziyi Xu
*Shanghai Jiao Tong University*

Mark Zhao
*University of Colorado Boulder*

Yuwei An
*Carnegie Mellon University*

Vikram Sharma Mailthody
*NVIDIA*

Scott Mahlke
*NVIDIA & University of Michigan*

Michael Garland
*NVIDIA*

Christos Kozyrakis
*NVIDIA & Stanford University*

## 摘要

长上下文大语言模型（ LLM ）使应用能够对数十万到数百万个 token 进行推理，但高效地服务这些工作负载具有挑战性。现代系统会缓存键值（ KV ）状态，并依赖跨 GPU HBM 、 CPU 内存和 SSD 的分层上下文缓存。我们表明，朴素的设计往往会受限于 I/O：碎片化的 KV 布局导致小规模传输，无法充分利用带宽；缓存加载会阻塞预填充；而忽略缓存加载延迟和延迟命中的调度器（延迟命中指缓存未命中期间对同一上下文的并发请求）会遭受严重的吞吐量下降。

我们提出 Strata，一个面向长上下文 LLM 服务的分层上下文缓存框架。Strata 引入了一种 GPU 辅助的 I/O 机制，将 GPU 与主机布局解耦，以实现高效的大规模传输；同时引入了一种缓存感知调度器，可缓解延迟命中、平衡批次以隐藏缓存加载延迟，并机会性地重叠互补工作。Strata 作为 SGLang 的一部分实现并已部署于生产环境，相比 vLLM-LMCache 吞吐量最高提升 5×，相比 NVIDIA TensorRT-LLM 最高提升 3.75× ，且不损害短上下文性能。

## 1 引言

大语言模型（ LLM ）代表了机器学习的一项重大进步，在理解和生成自然语言方面达到了卓越的水平。它们如今已被广泛采用，并正迅速朝着更强大的问题求解能力演进。这一演进中的一个显著趋势是上下文窗口的扩展，使 LLM 能够解析更长的输入提示。这促成了一系列需要理解大量文本的重要应用，包括编码助手 [6], 、检索增强生成（ RAG ） [27], 、文档分析 [49], 以及带有记忆模块的对话式 AI 智能体 [40]。领先的模型，包括 Google 的 Gemini 系列 [18], 、 Anthropic 的 Claude4 [7] 以及 Qwen 系列 [46], 已支持高达一百万个 token 的上下文窗口，预计很快将出现两百万 token 的窗口。其他前沿模型，如 DeepSeek-V3 [12], 、Llama-3.1 [3] 和 Llama-4 系列 [4], 也提供可观的上下文长度，通常在 128K 到 200K token 的范围内。

虽然长上下文带来了新的能力，但从头重新计算其上的注意力代价高昂得令人难以承受。缓存先前计算出的键值（ KV ）状态提供了一种实用的解决方案，因为这些前缀和来源在应用中经常被重复使用。这种技术通常被称为上下文缓存或前缀缓存 [5, 11, 17, 41, 53], ，可避免冗余的预填充计算，并显著降低响应延迟。然而，缓存 KV 状态的内存占用相当大。例如，40 GB 的 GPU 高带宽内存（ HBM ）对于 Llama-8B 只能容纳大约 0.3M 个 token，而这些容量可能很快被少量文档或几百轮对话消耗殆尽。GPU 内存的容量限制导致缓存频繁被驱逐，从而造成缓存命中率低和昂贵的重新计算。因此，生产系统采用分层缓存，将 KV 状态存储在 CPU 内存 [52], 、本地 SSD [14], 甚至远程内存池 [19,43] 中，以扩展容量并保留上下文缓存复用的优势。

然而，将大型缓存上下文传回 GPU 会引入主要的性能瓶颈。批量 KV 传输通常会导致内存停顿，直接影响首 token 时间（ TTFT ）并降低吞吐量。图 1 说明了这一效应：在使用 SGLang 将 KV 缓存卸载到 CPU 内存来服务 LooGLE 数据集 [28] 时，按照先前工作中报告的标准实践进行配置 [14],，74% 的预填充时间被阻塞在 KV 传输上（红色曲线），导致吞吐量最多降低 4×。在这些情况下，I/O 延迟而非计算成为主要的限制因素。我们观察到这种低效源于两个主要来源。

首先，KV 缓存布局导致 I/O 效率低下。随着上下文长度的增长，需要在内存层级之间（例如，CPU 内存到 GPU HBM ）移动的 KV 数据量大幅增加，然而当前系统仅能达到

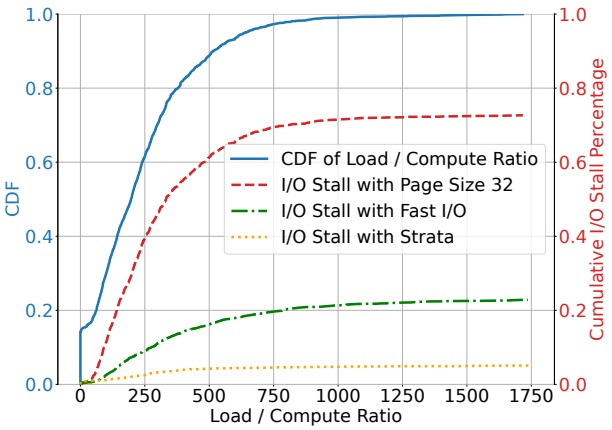



图 1：Qwen2.5-14B 在 LooGLE 数据集上的基准性能剖析。横轴表示每个预填充批次（ prefill batch ）的加载 / 计算比率（从 CPU 内存加载的 token 数相对于新输入 token 数）。右侧纵轴显示 I/O 停顿百分比，表示预填充执行时间中归因于 I/O 停顿的比例。完整基准测试详情见 §5.3 。

可用互连带宽的比例。正如我们在 §3.1 中详述的，这是因为当前系统采用 PagedAttention [26] 来减少 GPU 内存碎片化。然而，分页会导致 KV 缓存碎片化，分散在多个不连续的页上。这会产生许多小数据传输，有时仅有几 KB，无法饱和 PCIe 带宽。使用更大的页可以缓解碎片化并提高 I/O 效率，但会恶化缓存粒度并可能降低缓存命中率，正如我们在 §3.1 和 §5.3.2 中所展示的。

其次，当前调度器对缓存加载成本和延迟命中视而不见。现有系统 [26, 36, 53] 隐含地假设预填充新 token 所需的计算足以隐藏从较慢内存层级加载历史 KV 缓存的延迟。然而，随着上下文长度增长，这一假设不再成立：缓存加载时间可能超过预填充所需的计算时间，使系统处于加载受限而非计算受限状态。图 1 也凸显了这一效应。即使采用我们在 §4.2 中提出的优化 I/O 机制（该机制消除了小页传输的开销，绿色曲线），仍有高达 24% 的预填充执行时间停顿在缓存加载上。忽视这些 I/O 受限特征的调度器会生成不均衡的批次，无法有效隐藏缓存加载延迟。此外，在服务并发查询相同上下文的新兴智能体工作负载时，我们观察到 延迟命中现象 [8],，即针对同一长上下文的多个请求在初始缓存未命中仍在处理时到达。这降低了有效缓存命中率，尤其在高吞吐系统中，并导致长上下文的冗余计算，我们将在 §3.2 和 §5.3.4 中进一步详述。

为解决这些瓶颈，我们提出了 Strata，一个专为长上下文语言模型服务设计的分层上下文缓存框架，且在短上下文场景下不会出现性能下降。Strata 采用 GPU 辅助的数据传输来应对 KV 缓存碎片化问题，并将 GPU 的内存布局与其他内存层级解耦，从而提升跨层级的 I/O 效率。Strata 进一步通过缓存感知的请求调度来降低长上下文开销。它缓解延迟命中问题，构建均衡的批次，将充足的预填充计算与 I/O 延迟相匹配，并且在缓存加载停滞不可避免时，调度器会插入有用的补充任务（例如解码批次）以充分利用可用的计算资源。这些技术共同确保调度即使在高度可变的延迟预算下也能保持高效。

Strata 已集成到 SGLang [53], 一个被广泛采用的开源 LLM 服务框架中，并已在多家领先 AI 公司的生产环境中部署。我们在主流长上下文基准测试上、跨多种模型和硬件平台的评估表明，Strata 相比 vLLM-LMCache 一个最先进的开源分层上下文缓存解决方案，吞吐量最高提升 5×，相比 NVIDIA 的 TensorRT-LLM （一个高度优化的服务引擎）最高提升 3.75× ，且在短上下文场景下不会出现性能下降。

## 2 背景

### 2.1 长上下文 LLM 推理

LLM 推理分为两个阶段：预填充和解码。在预填充阶段，模型通常同时处理 (i) 来自用户查询的新 token 和 (ii) 上下文 token，这些上下文 token 来自文档或先前交互等来源。此步骤的中间输出称为 KV 缓存，对效率至关重要，因为它们允许后续 token 关注过去的 token，而无需重新计算所有先前的层。在解码阶段，模型自回归地生成 token，持续复用并扩展 KV 缓存。长上下文会放大两个阶段的成本。更长的前缀会增加预填充延迟，并且如果缓存状态被驱逐或不可用，重新计算该前缀会变得越来越昂贵。在解码期间，每个生成的 token 都会复用并扩展 KV 缓存，其大小随序列长度线性增长。因此，对于长上下文工作负载，KV 缓存的内存占用和数据移动可能成为主要瓶颈，使得高效的缓存复用、放置和驱逐对于高吞吐、低延迟的服务至关重要。

### 2.2 KVCache 的内存管理

受虚拟内存启发，PagedAttention [26] 通过使用动态的、基于页的分配方式，避免为 KV 缓存预留大的连续块。缓存被划分为

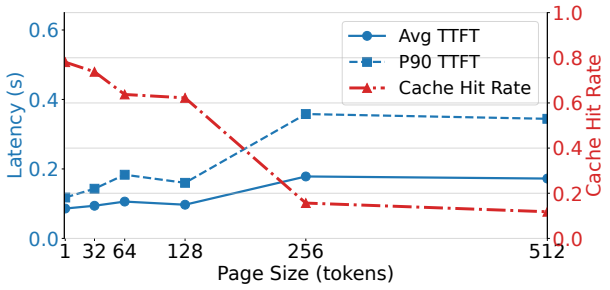



图 2：大页面尺寸会降低缓存命中率并增加 TTFT，基于 H200 使用 ShareGPT 数据集对 Mistral-24B 进行基准测试。

小的固定大小页面，保持逻辑顺序但可在内存中非连续放置，从而提高利用率。典型的页面大小很小 —— 例如，TensorRT-LLM 、 vLLM 和 SGLang 中分别为 32 、 16 和 1 个 token—— 而单个 token 的 KV 状态可能跨越数十千字节到数兆字节。虽然这种细粒度对计算内核来说是可管理的，但它对跨内存层级的数据移动带来了效率挑战，正如我们将在 §3.1 中讨论的那样。

## 2.3 LLM 服务中的上下文缓存

除了请求内复用之外，系统还通过使用前缀树或哈希映射等结构识别公共前缀，从而在请求之间利用上下文缓存，这些结构已被 OpenAI 和 Google 等提供商广泛采用 [26,43], 。为扩展容量，缓存被存储在较慢的层级中，如 CPU 内存 [16,22,52], 、分布式内存池 [19,29,43], 甚至磁盘 [14,21,30]。最近的系统，例如 CachedAttention [14],, 以逐层的方式将缓存加载与计算重叠，以最大限度地减少停顿，同时异步地将新生成的缓存备份到较低层级。

# 3 长上下文缓存的挑战

本文解决了为长上下文（即以预填充为主）工作负载管理大型上下文缓存的挑战。虽然这不是唯一的 LLM 场景（即存在短上下文、长生成或单轮工作负载），但长上下文工作负载构成了现实世界部署中的一类重要场景 [6,25,28]。接下来，我们将探讨长上下文工作负载中出现的系统挑战。

## 3.1 KV 缓存传输中的低带宽利用率

I/O 子系统可达到的吞吐量从根本上受限于利特尔法则所描述的关系。设 $\lambda$ 为 I/O 操作的到达率，$C$ 为并发 I/O 操作的平均数量，$L$ 为每次操作的平均延迟。根据利特尔法则，我们有 $C = \lambda \cdot L$。设 $X$ 表示持续的数据吞吐量（例如，以 GB/s 为单位），$S$ 为每次 I/O 操作的平均数据大小，则吞吐量可描述为 $X = \lambda \cdot S$。结合利特尔法则，我们得到 $X = C \cdot S/L$，描述了稳定状态下可达到的吞吐量。该等式强调，最大化数据吞吐量（$X$）需要高并发性（$C$）、大传输尺寸（$S$）或低延迟（$L$）。

为了在 CPU 和 GPU 内存之间传输数据，通常使用诸如 cudaMemcpyAsync 之类的异步操作来启用 GPU 的直接内存访问（DMA）引擎 [33]。然而，此场景中的延迟（$L$）包含不可忽略的 CPU-GPU 通信开销和调度延迟 [20]。虽然可以通过启动大量异步 I/O 操作来增加并发性（$C$），但实际限制来自可用的应用级并行性（例如，在 CPU 上）以及 GPU 驱动程序（例如，CUDA 驱动程序）和硬件内的队列容量 [33,44]。因此，增加传输尺寸（$S$）往往成为提高带宽利用率最实用的手段。饱和现代高带宽互连进一步印证了这一点；例如，要达到 PCIe 5.0 理论带宽的 75-80 %，需要传输尺寸（$S$）达到兆字节级别（即 1-2MB）。这一原则不仅限于 CPU-GPU 传输；在 SSD 或网络接口等其他介质上实现高吞吐量通常需要更大的传输尺寸 [9,13,32]。

**大页面的成本**。然而，与传输效率的要求相反，LLM 推理系统倾向于小粒度。较小的页面，例如在 1-32 个 token 的范围内，通常能带来更好的内存利用率 [26]，并且可以提高缓存命中率 [34],，因为缓存匹配是按页进行的。虽然增大页面大小（即，将更多 token 压缩到单个连续页面中，并有效增大 $S$）根据公式来看似乎可以提高带宽利用率，但它会给 LLM 推理带来显著的性能损失。为了证明这种权衡，我们使用流行的 ShareGPT 数据集对 Mistral-24B 模型进行了基准测试，在 SGLang 框架上将 KV 缓存页面大小从 1 变化到 512。如图 2 所示，增大页面大小会导致 KV 缓存命中率显著下降。这种退化直接导致平均 TTFT 和 P90 TTFT 大幅增加，在测试的最大页面大小下，二者分别最高上升 2× 和 2.9×。如图 10 所示，我们在长上下文基准测试场景中也观察到了类似趋势。

这种对小页面的偏好导致 KV 缓存操作的有效传输大小（$S$）非常小，从而导致 I/O 带宽严重利用不足。如图 3 所示，传输 8192 个 token 的 KV 缓存数据仅达到 PCIe 5.0 理论带宽的约 22%。页面大小设置为 32，这是先前关于分层 KV 缓存的工作中推荐的值 [14,19,21,52], 也是 vLLM 在 CUDA GPU 上支持的最大大小 [48]。这

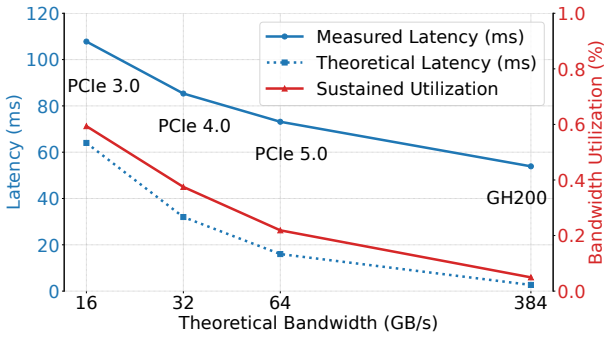



图 3：在不同平台上，将 Llama-3.1-8B 的 8192 个 token 的 KV 缓存（使用页大小 32 ）从 CPU 加载到 GPU 的延迟和带宽利用率。

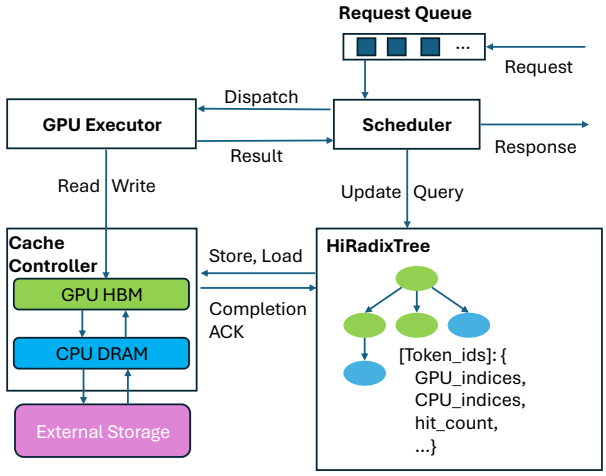



图 4：Strata 的系统架构

在互连带宽更高的平台上，利用率不足问题进一步加剧，在诸如 NVIDIA Grace-Hopper 等用 NVLink 替代 PCIe 并提供 6× 更高峰值带宽的系统上，利用率甚至低至 5%。传输效率与缓存收益之间的这一根本性权衡，凸显了对更高效 I/O 机制的需求，以兼得两者之长。

## 3.2 资源编排与延迟命中

LLM 服务引擎通常将 GPU 计算和 HBM 视为一等资源，因为子任务通常是计算密集型的（例如预填充中的稠密计算和注意力计算）或内存密集型的（例如解码）。随着分层 KV 缓存的引入，这一格局在很大程度上仍未改变。现有工作要么认为从 CPU 到 GPU 的 KV 缓存加载可以忽略不计 [14, 52], ，假设通过 PCIe 加载第 $N+1$ 层缓存数据的延迟可以通过将 I/O 与第 $N$ 层的计算重叠来有效隐藏；要么在成本合理时选择重新计算 [23]。然而，当服务具有长缓存上下文但需要预填充的新 token 相对较少的请求时，这一假设便不再成立。一方面，批量 KV 缓存传输的延迟可能超过层级计算所能隐藏的程度；另一方面，随着上下文长度增长，重新计算的成本越来越高，使其成为一种不具吸引力的替代方案。因此，逐层重叠的预填充可能退化为受 PCIe 带宽限制的任务。图 1 展示了这一瓶颈：即使采用达到理论 PCIe 带宽 75% 的 I/O 机制，停顿仍占预填充执行时间的高达 24%。

使这一挑战更加复杂的是 延迟命中现象 [8]，此前已在网络社区中研究过。当针对同一数据对象的多个请求在初始缓存未命中仍在解决时到达，后续请求实际上被当作缓存未命中处理，尽管如果第一个请求已完成，它们本应是缓存命中。我们在 LLM 中观察到了类似效应服务：在高流量下，针对同一上下文（或其前缀）的多个请求可能在很短的时间窗口内到达，这在新兴的智能体工作负载中尤为常见，因为并发尝试往往会复用相同的上下文。例如，在 Mooncake 智能体工具追踪中，[43],38% 的请求与至少一个在一秒窗口内到达的其他请求共享至少 6k token 的前缀（不包括系统提示），这使得在高吞吐服务系统中延迟命中极有可能发生。如图 7 所示，当此类请求被调度为并发执行时（无论是在同一批次内还是跨数据并行实例），就会发生冗余的预填充计算。现代服务引擎中广泛采用的异步调度器 [47,55], 会加剧这一问题，这些调度器会在当前批次完成之前准备下一批次，将缓存未命中的解决窗口延长至一个批次的完整执行时间，从而使延迟命中更有可能发生。在长上下文场景中，这种影响尤为严重，因为重计算成本和缓存未命中解决时间都会以不利的方式扩展。

这些观察结果凸显了重新思考 LLM 服务中调度策略的必要性。调度器必须明确将 CPU-GPU 带宽视为一等资源，在对请求进行批处理时平衡计算与数据传输，更仔细地使 I/O 与计算重叠，并缓解延迟命中，以避免冗余工作，同时维持吞吐量。

# 4 Strata 的设计与实现

## 4.1 概述

受 §3 中所讨论挑战的启发，我们构建了 Strata，一个包含两个关键组件的系统。Strata 缓存控制器（ §4.2 ）管理整个内存层次结构中的数据平面元素。它引入了一种优化的 GPU-CPU 数据

传输机制，并管理 KV 缓存内存布局，以支持如 §3.1 中所激励的高效小页传输。Strata 调度器 （ §4.3 ）实现了控制平面，以如 §3.2 中所激励的缓存资源感知方式智能地调度请求。它引用了一个 *HiRadixTree*，这是对 SGLang 的 RadixTree [53], 的扩展，有效地充当页表，并存储关于每个 KV 缓存页的元数据。

图 4 展示了 Strata 架构。当请求被提交时，它会进入请求等待队列。在正在进行的批次执行期间，调度器 持续估计可用系统资源和排队请求的资源需求，并选择一个子集以形成下一批次。然后，调度器 将此批次发送到 GPU 执行器，并向 缓存控制器发起 KV 缓存加载请求。在预填充批次执行期间，GPU 执行器使用 CUDA 事件与 缓存控制器 同步，以确保在执行使用某一层的 KV 缓存之前该层的 KV 缓存可用。一旦预填充完成，预填充的请求通过连续批处理 [51] 合并为一个统一的解码批次。Strata 采用预填充 - 解码（ P-D ）共置设计，在同一 GPU 上按时间交替执行预填充批次和解码批次，并遵循 SGLang 的做法，优先执行预填充批次以获得更短的响应时间（ TTFT ），并形成更大的解码批次以获得更高的吞吐量。最后，缓存控制器 主动管理任何 KV 缓存页到较低内存层级的备份和驱逐，且以异步方式进行。

## 4.2 高效的 KVCache I/O

为解决 §3.1 中讨论的局限性，受计算机体系结构社区 [33,44], 中既有实践的启发，Strata 利用 GPU 辅助 I/O 在 CPU 和 GPU 内存之间传输 KV 缓存页，从而对小而碎片化的数据实现低延迟 I/O。具体而言，GPU 辅助 I/O 作业不是通过反复调用标准的 *cudaMemcpyAsync* API 来进行小数据传输，而是通过启动一个 CUDA 内核来运行。该内核会生成数千个线程。每个线程负责从源（ GPU 全局内存或 CPU 注册的固定内存）加载一小块数据到其本地寄存器文件中，然后将此数据流式传输到目标（ GPU 全局内存或注册的 CPU 固定内存）。

GPU 辅助 I/O 具有若干优势：首先，它能够实现**增强的并发性**（ $C$ ）：GPU 提供大规模、高性价比的并行能力，可支持数千个并发 I/O 操作，而 CPU 通常仅支持数十个。其次，它**兼容小传输**（高效 $S$ ）：在大多数架构上，高效 GPU 辅助 I/O 所需的粒度仅为 128 字节 [39],，这对于单页 KV 缓存（千字节级）而言已足够精细，从而无需为提高效率而增大页面大小。最后，它允许**灵活的内存布局**：由于 I/O 内核中的轻量计算几乎是

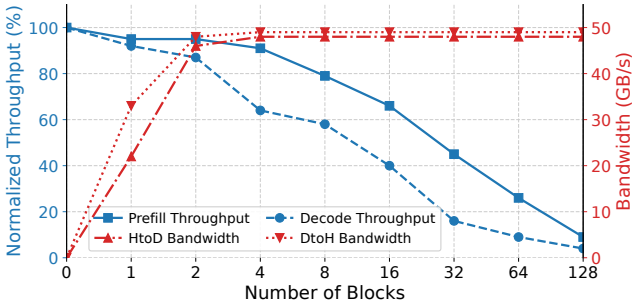



图 5：性能干扰与分配给 KV 缓存 I/O 内核的资源之间的关系。分别在并发运行 Strata 的 I/O 内核与预填充阶段（两个请求的批次，每个输入 4k ）以及解码阶段（ Llama-3.1-8B，16 个请求的批次，每个输入 4k ）时进行测量。

几乎零成本，GPU 与 CPU 内存之间的布局转换可以以可忽略的开销完成，从而实现灵活且高效的数据组织（详见 §4.2.1 ）。

然而，如先前工作 [20], 所强调的，GPU 辅助 I/O 面临的一个挑战是与其他内核共同运行时的运行时干扰。在没有专用硬件处理细粒度 I/O 任务的情况下，GPU 线程会消耗宝贵资源，例如寄存器文件和执行周期，并可能导致缓存污染。先前工作 [31] 还表明，GPU 硬件调度器往往难以有效管理这种资源争用，可能降低 I/O 操作和并发计算内核的性能。

我们观察到，高效的数据传输无需独占整个 GPU。Strata 采用启动少量大型 CUDA 块的策略，以激励 GPU 硬件调度器将这些 I/O 内核限制在少量流式多处理器（ SM ）上，最少可至 1 个。这种定向分配，结合绕过缓存从而减轻污染的底层指令，可最大限度地减少对并发工作负载的干扰。此外，借助 ROCm 后端 [2],，这些内核实现也与 AMD GPU 兼容。为平衡整体效率所需的资源，我们在 NVIDIA H200 GPU 上进行了将 I/O 内核与预填充和解码内核共同运行的微基准测试。如图 5 所示，仅使用两个各含 1024 个线程的 CUDA 块，Strata 即可实现 48 GB/s 的传输吞吐量，同时对预填充造成低于 5% 的性能下降，对解码造成 10% 的性能下降。基于这些结果，我们选择两个块作为从 CPU 向 GPU 加载数据（关键路径操作）的默认配额，并选择一个块用于从 GPU 向 CPU 备份数据（非关键路径），此时带宽已足够且必须最小化开销。我们的端到端评估证实，此配置可维持高 I/O 带宽，同时保持

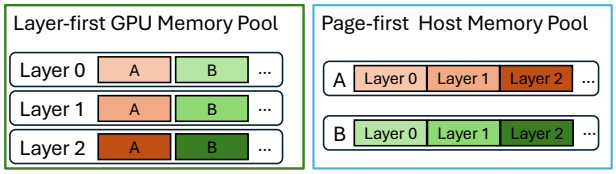



图 6：层优先与页优先布局

整体性能影响低于 5%，表明经过精心调优的 GPU 辅助 I/O 既能高效又不具侵入性。

### 4.2.1 超越主机内存的数据管理

当涉及外部存储时，缓存控制器会在存储层检测到缓存命中时，机会性地将数据从存储预取到主机内存中。预取延迟与请求的排队延迟重叠。一旦调度器选择该请求执行，缓存控制器会终止任何正在进行的预取，并使用主机或 GPU 内存中已有的缓存。这种默认的尽力而为策略的动机在于，存储层相对于主机 -GPU 传输的延迟显著更高且更难以预测，为此 Strata 采用了一种同步的、逐层重叠的方案。Strata 调度器还可以配置为将正在进行预取的请求保持到预取完成（等待完成策略）或直到指定超时（超时策略），具体取决于部署场景。

此外，如 §3.1 中所论述的，由碎片化内存布局导致的数据传输低效问题也延伸到其他存储介质。除了小页面之外，LLM 服务系统还倾向于在 GPU 内存池中采用层优先的内存布局（如图 6 所示），因为它与 LLM 计算的逐层特性相一致。具体而言，层优先布局将第 0 层的所有 token 连续存储，然后是第 1 层的所有 token，依此类推，使得某一层的注意力内核可以将其 K/V 作为一个连续跨度读取。这对于计算是理想的，但对于传输却是最差的：单个逻辑页面（一个 token 跨所有层的 K/V ）被分散成 L 个不连续的片段，每层一个，每个仅几 KB。而传输友好的页优先布局则将页面的所有层连续存储，产生大的连续块以饱和互连带宽，但这会迫使每个注意力内核经过额外的地址间接层。通过利用 GPU 辅助 I/O，Strata 通过实现计算友好与传输友好布局之间几乎零开销的即时转换来解决这一冲突。要执行布局转换，线程只需对其分配的偏移量应用一个额外的算术运算来计算正确的目标地址。此操作的开销可忽略不计。

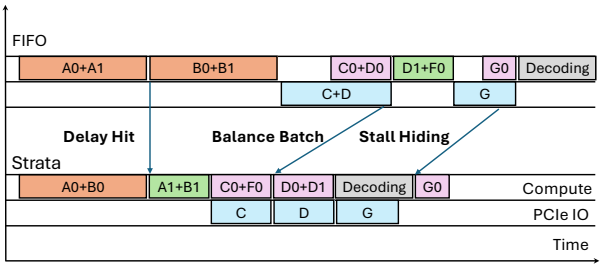



图 7：Strata 中的调度策略。橙色块表示会产生缓存未命中的预填充批次；绿色块表示设备内存中的缓存命中；紫色块表示主机内存中的缓存命中；蓝色块表示数据传输。一个解码批次以灰色显示。$X_n$ 表示使用上下文 $X$ 的第 $n$ 个请求。

如图 6 所示，该能力解耦了跨内存层次结构的布局要求：GPU 可以保持其计算友好的层优先布局，而其他介质（如主机内存和外部存储）可以采用页优先布局，通过更大的连续数据块来最大化传输效率。在 §5.3.5 中，我们展示了这种解耦布局策略如何显著减少数据加载时间。

## 4.3 缓存感知调度

如 §3.2 所述，调度器的目标是通过避免延迟命中和加载停顿来最大化缓存收益。调度器通过三个阶段实现这一目标。首先，它识别可能易受延迟命中影响的请求，并将执行**推迟**到延迟命中解决之后。其次，它**制定一个均衡批次**，旨在将（来自主机内存的）加载与足够的计算配对，以隐藏加载延迟。最后，在批次仍然受加载限制的情况下，调度器**通过在气泡中插入有用的计算来隐藏 I/O 停顿**。注意，虽然 SGLang 的核心调度器是用 Python 实现的，但我们用 C++ 实现对其进行了增强，以最小化这些调度策略的开销。

### 4.3.1 延迟命中时的推迟

如 §3.2 所述，延迟命中可能在两种情况下导致冗余计算：(i) 当多个共享同一缓存未命中的请求被调度到同一批次中时（图 7），以及 (ii) 当请求的执行被异步准备，而未意识到相应的上下文缓存仍在计算中时。为了跟踪潜在的延迟命中，我们在 HiRadixTree 中引入了瞬态节点。与标准节点类似，它们使用 token ID 作为遍历键，但

**Algorithm 1** Balanced Batch Formation

```
1:  procedure AddBundleHit(Q, B)
2:      for each r in Q do
3:          if B.is_bundle_hit(r) then
4:              B.add(r); Q ← Q − r
5:  function BatchFormation(Q)
6:      B ← Batch(); D ← []
7:      B.add(Q.pop(0)); AddBundleHit(Q, B)
8:      while |Q| > 0 and ¬B.is_full() do
9:          r ← Q.pop(0)
10:         if B.loading_bound(r) then
11:             D.append(r)
12:         else
13:             B.add(r); AddBundleHit(Q, B)
14:     for each r in D do
15:         if B.is_full() then break
16:         B.add(r)
17:     return B
```

它们不指向内存索引，而是携带两种标记之一：in-queue，表示某个请求正在引用新的上下文；in-flight，表示对应 token 的缓存正在计算中。在遍历请求队列时，Strata 会按需插入标记为 in-queue 的瞬态节点。如果某个请求与现有瞬态节点匹配，则该请求会被推迟到下一轮调度，但会被放置在等待队列的前端，以便从即将变为热点的缓存中获益，并尽量减少其对 TTFT 的影响。

当某个请求进入执行阶段时，其关联的瞬态节点会被标记为 in-flight。完成后，这些节点会被转换为标准节点，其索引指向已缓存在内存中的就绪上下文。为避免不必要的推迟，Strata 使用一个可配置阈值：只有当瞬态节点上的 token 匹配数超过该值时，请求才会被推迟。我们使用默认阈值 100 个匹配 token，这样既能避免推迟较短的偶然前缀匹配，又能捕获长上下文延迟命中。

#### 4.3.2 均衡批次形成

在移除易受延迟命中影响的候选请求后，调度器会选择请求来组成下一个预填充批次。在大多数 LLM 服务引擎中，[26, 53]，批次形成默认遵循先进先出（FIFO）策略，即按到达顺序取请求，直到批次已满（要么达到预设的 token 限制，要么耗尽 GPU 内存）。为解决 §3.2 中讨论的加载受限问题，Strata 引入了一种新的批次形成机制，以平衡数据加载与充分计算。图 7 展示了一个示例，包含请求 C0 和 D0 的批次将需要加载两个上下文，从而导致加载停顿。相比之下，形成不同的批次（C，F）可以使 C 的加载被重叠执行。

另一个有用的案例是批次（D0，D1）：由于它们共享相同的上下文，将它们组成批次不仅能平衡加载，还能减少 GPU 内存使用和片上带宽压力，从而进一步提高效率。我们将这种情况称为 *bundlehit*，即 *delay hit* 的对应情形。

该过程在算法 1 中详细描述。在每个批次形成之前，调度器使用 HiRadixTree 获取每个请求的负载和计算需求。在批次形成期间，当调度器遍历队列时，它会检查添加请求是否会使批次达到加载受限上限（第 10 行中的 ‘loading_bound’），该上限定义为聚合负载与计算量的比率。当该比率超过阈值时，批次被视为加载受限。该阈值取决于硬件和模型，因此可以单独进行性能分析；在实践中，Strata 使用默认比率 100，对应于图 1 中所示开始出现停顿的点。如果请求能够加入批次而不会使批次变为加载受限，则将其加入批次，然后调度器将遍历剩余请求，优先添加与其捆绑命中的请求。否则，该请求被移至降优先级列表。如果直到队列末尾批次仍未满，调度器会用降优先级请求补充批次（第 16 行）。为防止饥饿，降优先级请求保留其原始顺序，并且每次批次形成总是从队列中的第一个请求开始。

#### 4.3.3 使用气泡填充隐藏加载停顿

即使采用均衡批处理，某些批次仍可能处于加载受限状态。调度器的最终策略是气泡填充，即将加载停顿与有用计算重叠。图 7 展示了一个示例，当请求 G0 需要长上下文加载时，调度器推迟已准备好的预填充批次的計算，转而向模型执行器发出一个解码批次，使其与上下文加载并发运行。该策略补充了 SGLang 默认的预填充优先策略（如 §4.1 中所讨论），在预填充和解码之间进行选择时提供了一定的灵活性，以提高整体利用率。尽管解码批次也是 I/O 受限的，但它们主要饱和 HBM 带宽，而加载任务饱和 PCIe 带宽。这种区别使得两种操作能够以最小的资源争用重叠进行。也可以插入预填充批次来填充气泡，这更适用于 P-D 分离系统 [54]。

### 4.4 缓存控制器与策略

如图 4 所示，缓存控制器负责监管各内存层级之间的数据移动。当为新批次准备执行时，调度器通过 HiRadixTree 启动将 KV 缓存加载到 GPU。随后，在执行期间，GPU 执行器会咨询缓存控制器，以确保特定的 KV 缓存数据

在继续执行后续层之前，ci c 层已可用。在预填充或解码请求完成后，调度器启动将 KV 缓存存储到较慢的内存层级，根据指定的缓存写入策略备份特定的 KV 缓存数据。

单一的写一次策略不足以实现分层上下文缓存，因为不同的服务负载在写入带宽、缓存容量、持久性和未来重用之间有着不同的权衡。立即写入每个生成的 KV 页可以最大化重用和持久性，但对于一次性请求可能会浪费主机内存和写入带宽。仅在驱逐时写入可以最小化写入流量，但当 GPU 内存面临压力时，可能会在关键路径上增加阻塞工作。因此，Strata 提供了三种写入策略，对应三种部署场景。第一种，*写回*，仅在 KV 缓存令牌即将被驱逐时才进行备份。这种方法适用于资源受限的环境，因为它最小化了带宽和容量使用，尽管它可能引入额外的运行时阻塞。第二种策略，*写穿透*，每次生成新的 KV 缓存时都会启动备份，非常适合所有输入和生成内容都必须持久存储的对话场景。第三种也是默认策略是*选择性写穿透*。采用这种方法时，HiRadixTree 中的每个节点都关联一个计数器，每次访问该节点时递增。仅当此计数器超过可配置的阈值时才触发备份。该策略提供了最大的灵活性：将阈值设为一使其等同于写穿透策略，而更高的值可以减少内存占用，特别是对于单轮问答任务等场景。当写入带宽充足时，默认阈值设为 2，但可以增加以适应更紧的资源约束。对于所有内存层级，最近最少使用（ LRU ）算法作为默认驱逐策略。

# 5 评估

## 5.1 方法

**测试平台**。我们在三个平台上评估 Strata 和基线。*H200* 平台是一个配备 8 块通过 NVLink 互连的 NVIDIA H200 GPU 、一颗 Intel Sapphire Rapids CPU 和 1.6TB DRAM 的节点。每块 GPU 通过 PCIe 5.0 x16 链路连接到 CPU，提供高达 64 GB/s 的峰值带宽（单向）。*H20-storage* 是一个配备 8 块 NVIDIA H20 GPU 和一块 Intel P5510 NVMe 驱动器的节点，提供高达 7 GB/s 的磁盘读取带宽。*GH200* 平台是一个 GH200 Grace Hopper 超级芯片 [35] 节点，包含一块 NVIDIA H100 GPU 与一颗 NVIDIA Grace 64 核 ARM CPU 集成。*GH200* 系统配备 464 GB 的 LPDDR5X DRAM，为 CPU 提供高达 384 GB/s 的内存带宽（单向）。

**基线**。我们将 Strata 与以下最先进的基线进行比较。*vLLM* [26] 是一个流行的开源服务引擎。此外，*vLLM-LMCache* 通过 LMCache [30] 的官方社区扩展在 vLLM 上实现了分层缓存。在我们的基准测试中，我们使用了 vLLM v0.8.5 和 LMCache v0.2.1。LMCache 的块大小默认设置为 256，vLLM 的页大小按照先前工作设置为 32 [14]。*TensorRT-LLM* [36] （ TRT-LLM ）是 NVIDIA 推出的开源服务库，专为 NVIDIA GPU 优化。此外，*TRT-LLM-HiCache* 通过其自动 CPU 内存卸载功能在 TensorRT-LLM 之上实现了分层缓存。我们在基准测试中使用了 TensorRT-LLM v0.17.0，页大小也默认设置为 32。*SGLang* [53] 是一个开源服务引擎，可提供与 vLLM 相当的性能，同时具有更轻量、更可定制的架构。为了与 Strata 进行直接比较，我们实现了 *SGLang-HiCache*，它采用最先进的逐层 KV 缓存传输重叠和分层缓存实现，使用 *cudaMemcpyAsync* 传输，这与包括 CachedAttention [14], 、 Pensieve [52] 和 FlashGen [21] 在内的先前工作一致。我们对所有三个系统均使用了 SGLang v0.4.5。我们将 *SGLang* 和 Strata 的页大小设置为 1 （ SGLang 的默认值），并将 *SGLang-HiCache* 的页大小设置为 32，以与其他分层缓存基线保持一致。

**模型**。我们使用了三个具有长上下文能力的流行开源 LLM，涵盖小、中、大三种规模：Llama-3.1-8B-Instruct [3] （ 128k 上下文窗口）、 Qwen2.5-14B-Instruct-1M [46] （ 1M 上下文窗口）和 Llama-3.1-70B-Instruct [3] （ 128k 上下文窗口）。我们使用单块 GPU 服务 8B 和 14B 模型，并使用 4 块 GPU 以张量并行方式配置来服务 70B 模型。

**数据集**。我们基于三个长上下文数据集构建工作负载。*LooGLE* [28] 包含来自 arXiv 、维基百科和电影 / 电视脚本等多种来源的长文档。在我们的基准测试中，我们使用其维基百科部分，该部分提供了与文档配对的长查询和短查询。*NarrativeQA* [25] 是一个具有影响力的长上下文数据集，用于测试模型的阅读理解能力，其上下文示例甚至比 LooGLE 更长。由于测试模型的上下文窗口限制，我们过滤了超过 128k token 的文档，并从剩余文档中采样了 50 个文档。这两个数据集反映了经典的 RAG 用例，其中大量上下文随时间被多个用户反复查询，如同针对技术手册的问答系统 [15]。*ReviewMT* [45] 是一个多智能体对话数据集，其中智能体模拟审稿人进行对话

| | LooGLE | NarrativeQA | ReviewMT | ShareGPT |
|---|---|---|---|---|
| avg. in | 21613 | 54797 | 17708 | 680.9 |
| avg. out | 15.60 | 13.00 | 208.3 | 260.9 |
| # contexts | 105 | 50 | 100 | - |
| # queries | 2410 | 1461 | 1092 | 200869 |

表 1：数据集统计。

关于技术论文质量以做出最终决策。这代表了一个典型的智能体工作流，涉及长上下文以及相比其他数据集相对较长的输出。我们还纳入了一个数据集来评估 Strata 在短上下文场景中的性能。*ShareGPT* 是一个流行的对话数据集，由来自数千名用户的大量相对较短的对话历史组成，并曾被用于先前的分层 KV 缓存研究 [14,52]。表 1 总结了这些数据集的特征。

由于这些数据集中无法获得单个查询的时间戳，我们遵循先前的工作 [21, 52]，使用泊松分布模拟查询到达，以在不同请求速率下对系统进行基准测试。对于对话基准（ReviewMT 和 ShareGPT），我们保留跨对话轮次的依赖关系。与 Pensieve [52], 中的方法一致，我们在 LLM 响应与用户在 ShareGPT 中的后续查询之间插入 60 秒的 “思考时间”。对于长上下文基准，查询从数据集中随机采样。在 §5.3.3 中，我们进一步考察了不同工作负载模式下的性能特征。为避免基准测试超时和无限制的客户端排队，我们将最大在途请求数限制为 128。请注意，这是并发上限，而非固定批大小，因为实际预填充批大小由每个服务引擎的调度器、令牌预算和可用 GPU 内存决定。GPU 内存按照每个服务引擎的默认策略进行分配，以确保公平性和性能。一个例外是 ShareGPT 数据集，我们将 GPU 内存限制在约 500K 令牌，以突出分层缓存基线的行为。对于利用 CPU 内存的缓存配置，我们分配 1 TB 系统 DRAM 作为固定内存（由于平台限制，在 GH200 上为 400 GB）。大多数基准测试中不使用磁盘存储，除了 §5.3.5，这是由于基线系统的支持有限。

## 5.2 端到端性能比较

Strata 旨在通过降低响应延迟和提高整体吞吐量来改进长上下文服务。因此，我们使用两个主要指标来评估系统：平均首令牌时间（TTFT）和输出令牌吞吐量。TTFT 反映查询响应时间，是用户体验的关键决定因素，而输出令牌吞吐量是表征 LLM 服务系统性能的广泛采用的指标 [1]。

### 5.2.1 Strata 在长上下文工作负载上的性能与最先进的 LLM 服务系统相比如何?

图 8 展示了在三种模型和四个数据集上，各系统在不同请求速率下所达到的令牌吞吐量及相应的平均 TTFT。首先，我们观察到分层缓存对于高性能长上下文服务至关重要。在所有三种模型上，非分层缓存解决方案在 LooGLE 上表现不佳。没有分层缓存，长上下文工作负载会迅速耗尽 GPU 内存容量，导致预填充期间频繁出现缓存未命中。这反过来又触发重复重计算，导致低吞吐量和高延迟。相比之下，配备分层缓存的系统实现了显著更高的吞吐量和更低的延迟，通过利用 CPU 内存持续达到约 95% 的缓存命中率。

其次，Strata 相较于现有的分层缓存解决方案带来了显著改进。对于 LooGLE 上的 Llama-8B，在相同 TTFT 下，Strata 相比 SGLang-HiCache 、 vLLM-LMCache 和 TRT-LLM-HiCache 分别实现了高达 3.2×、 2.6× 和 1.9× 的吞吐量提升。Qwen-14B 也观察到类似增益，Strata 实现了高达 3.9×、 2.1× 和 1.9× 的提升；对于 Llama-70B，增益达到 5×、 5× 和 3.75×。在 ReviewMT 上也观察到一致的趋势。尽管更长的解码降低了预填充时间的主导地位，但使用 Llama-8B 时，Strata 相比 SGLang-HiCache 提升了 1.7×, 相比 vLLM-LMCache 提升了 2.3× ，相比 TRT-LLM-HiCache 提升了 2.3×。这些性能增益源于 Strata 增强的 I/O 效率和调度机制，§5.3.1 将进一步分析。

### 5.2.2 Strata 在热缓存下的表现如何?

NarrativeQA 的平均上下文长度要长得多，导致了一个广泛的预填充阶段，这使得非分层缓存解决方案变得不切实际。由于该场景呈现出最高的内存压力，我们对该实验进行了增强，以更好地理解稳态性能特征（即缓存层次结构被填满之后）[1]。在此设置中，我们首先通过为评估集中的所有上下文预计算 KV 缓存来预热 CPU 内存，然后刷新 GPU 内存上的 KV 缓存以设置初始状态。图 8 的第二行报告了在预热后重新启动工作负载后的吞吐量 - 延迟曲线。我们报告了 Strata 、 *SGLang-HiCache* 和 *vLLM-LMCache*，因为 *TRT-LLM-HiCache* 不支持预热。在此设置下，与 vLLM-LMCache 相比，Strata 在 Llama-8B 、Qwen-14B 和 Llama-70B 模型上分别实现了高达 2.3×、2.6× 和 2.5× 的吞吐量。

### 5.2.3 Strata 在短上下文下的表现如何?

Strata 是专为长上下文工作负载而设计的。为了解 Strata 在短上下文工作负载上的性能，图 8 的最后一行展示了基线系统在 ShareGPT 数据集上跨三个模型的平均 TTFT。请注意，底层 SGLang 引擎相较于基础引擎表现出轻微的性能劣势

[1]NarrativeQA 在不进行预热的情况下取得了与 LooGLE 工作负载相似的结果。由于篇幅限制，我们省略了该场景的结果。

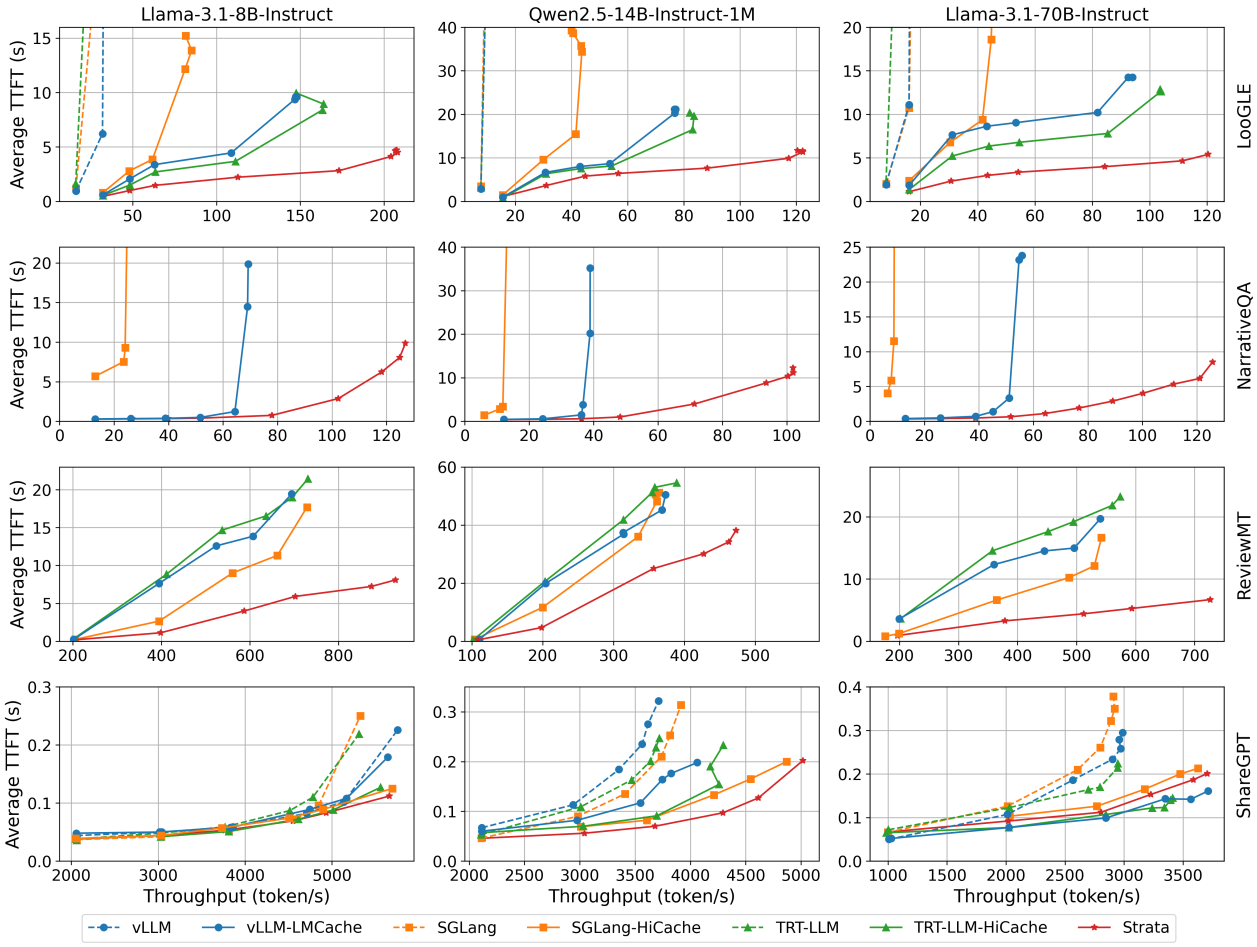



图 8：在 H200 上的端到端基准性能比较。

由于内核差异，vLLM 和 TRT-LLM 在 Llama-8B 和 -70B 模型上的表现有所不同。考虑到这一点，Strata 在短上下文工作负载上展现出与其他最先进系统相当的性能。

## 5.3 深入性能分析

在本节中，我们进行了多项分解分析，以更好地理解 Strata 所提供的性能优势。除非另有说明，所有分析均在 H200 平台上使用 Qwen2.5-14B 模型和 LooGLE 基准进行。

### 5.3.1 高效的 I/O 和调度能为 Strata 带来多少收益?

图 9 展示了 Strata 与三个基线相比的吞吐量 - 延迟曲线。在 *SGLang-HiCache* 的基础上，我们构建并评估了三个消融变体：*Strata-IO*，它集成了 §4.2 中的 GPU 辅助 I/O 机制，*Strata-Schedule-Only*，它应用了调度策略来自 §4.3，以及 *Strata-IO-LPM*，其集成了最长前缀匹配（LPM）策略 [53]。

结果表明，*Strata-scheduling* 和 *Strata-IO* 两个组件均显著改善了基线分层设计，分别实现了高达 1.8× 和 2.3× 的更高峰值吞吐量。每个组件从不同角度缓解了加载停滞问题。在低请求速率下，*Strata-scheduling* 往往比 *Strata-IO* 带来更大的收益，因为较小的批大小产生的 I/O 压力更轻，可以通过高级调度更有效地缓解。然而，随着请求速率增加，I/O 子系统成为主要瓶颈，使得 GPU 辅助的 I/O 机制对于维持高吞吐量至关重要。

我们进一步将 *vLLM-LMCache* 和 *TRT-LLM-HiCache* 直接与 *Strata-IO* 进行比较，因为三者均采用 CUDA 内核来加速 KV 缓存 I/O。如图 9 所示，它们在低请求速率下性能相当，但 *Strata-IO* 随着请求速率上升仍保持更高吞吐量，表明在大规模下更有效地缓解了干扰。我们还将 Strata 与 *Strata-IO-LPM* 进行了比较，其

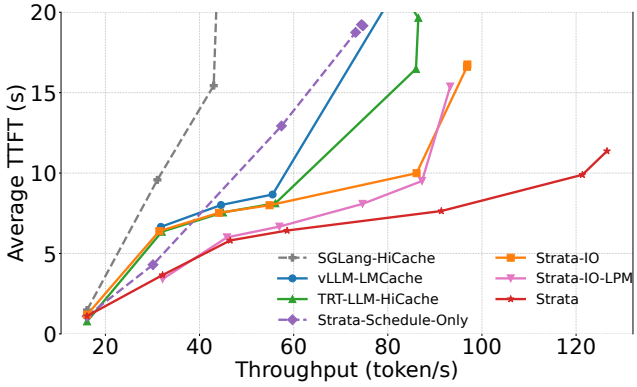



图 9：Strata 的 I/O 与调度分解。

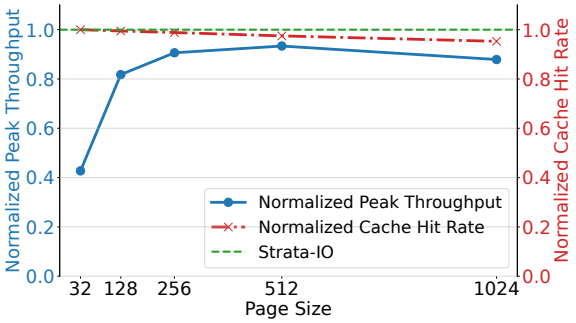



图 10：不同页面大小下 Strata-IO 与 SGLang-HiCache 的性能比较。

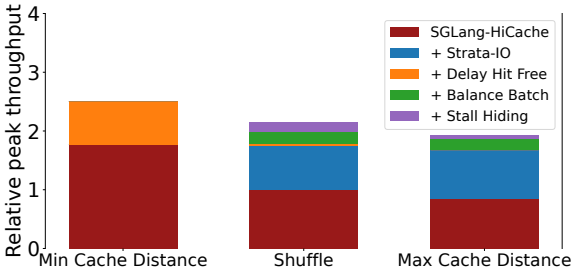



图 11：Strata 在不同工作负载模式下的优化贡献分解。

提高了设备端页面的复用次数，从而间接降低了主机侧加载压力，并在低请求速率下提升了性能。然而，在较高请求速率下，由于缓存驱逐更加频繁，它无法维持性能增益。相比之下，Strata 能够持续带来改进，因为它在设计中明确考虑了带宽资源。

### 5.3.2 Strata 能否减轻选择页面大小的负担?

正如 §3.1 中所讨论的，页面大小在缓存命中率、 I/O 效率和整体性能之间引入了固有的权衡。这使得选择最优页面大小成为一个实际但并非易事的问题。Strata 使传输效率对页面大小的敏感度大大降低，从而减少了仅为获得高效传输而选择大页面的需求。为验证这一说法，图 10 报告了 *SGLang-HiCache* 在不同页面大小下的峰值吞吐量，并归一化到 *Strata-IO*。对于 *SGLang-HiCache*，增大页面大小最初会提升吞吐量，因为更大的页面减少了加载停顿。然而，超过某个阈值后，随着缓存命中率下降，吞吐量会下降。即使在其最佳性能设置下（页面大小为 512），*SGLang-HiCache* 也仅达到 *Strata-IO* 性能的 93%，主要原因是缓存命中率低了 2.4%。

### 5.3.3 Strata 能否适应变化的缓存距离?

缓存距离是用于测试缓存系统性能的跟踪的重要属性。为进一步增大我们工作负载的缓存距离，我们基于 LooGLE 数据集生成了两个额外的工作负载。除了我们原始的（ shuffle ）工作负载外，我们还创建了一个最小缓存距离工作负载，其中共享相同上下文的请求被排在一起，以及一个最大缓存距离工作负载，其中共享相同上下文的查询被均匀分布在队列中。如图 11 所示，我们观察到，在最小缓存距离下，由于，无需分层缓存完美的局部性。在此场景下，延迟命中缓解通过提高有效缓存命中率获益最大，将峰值吞吐量提升了 42%。然而，延迟命中缓解在最大缓存距离场景下没有带来任何收益，因为相似请求之间的高距离自然降低了延迟命中现象发生的可能性。相比之下， §4.2 中介绍的 I/O 效率机制在乱序和最大缓存距离场景下分别带来了 76 % 和 95% 的提升，因为更大的缓存距离导致更多缓存命中到 CPU DRAM。在此基础上，平衡批处理进一步分别引入了 11% 和 12% 的峰值吞吐量提升。停顿隐藏进一步为乱序和最大缓存距离分别贡献了 8% 和 3%，由于方差更高，对乱序模式的收益更大。

### 5.3.4 是什么导致了延迟命中?

除了 §5.3.3 中讨论的请求模式之外，两个系统级因素在很大程度上决定了延迟命中效应的严重程度：缓存解析时间和系统吞吐量（以及由此决定的请求到达率）。如图 12 所示，更长的缓存解析时间（由预填充和调度延迟驱动）和更高的吞吐量水平都会增加因延迟命中而导致的缓存未命中数量。对于 1k token 输入，典型的缓存解析时间从数百毫秒到数秒不等，具体取决于模型和硬件。注意，我们使用 Moon- 的模拟执行来研究这一效应

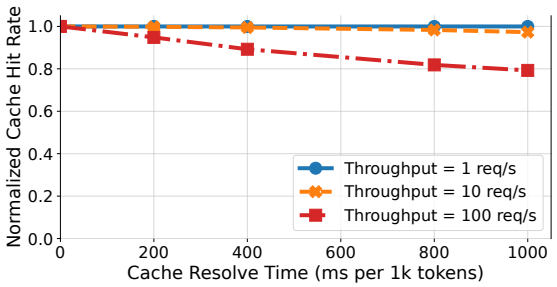



图 12：在不同缓存解析时间和吞吐量下，延迟命中对 Mooncake Tool-Agent 轨迹的影响。不同的吞吐量通过缩放查询时间戳来实现。

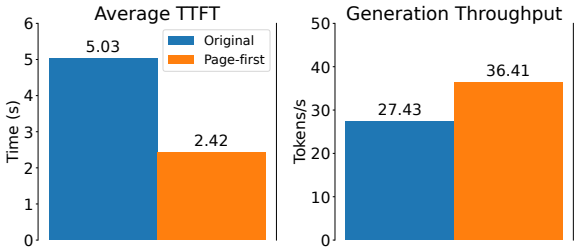



图 13：在 8×H20 GPU 上，DeepSeek-V3 的解耦内存布局带来的端到端性能提升，页面大小为 32，请求速率为 12 req/s。

cake 轨迹，使用实际上无限的缓存（以消除驱逐效应），因为我们的测试平台无法维持足够高的吞吐量。

### 5.3.5 解耦内存布局是否有利于磁盘缓存?

GPU 辅助 I/O 可在主机内存（和磁盘）中实现页优先布局，而无需更改驻留 GPU 的布局（ §4.2.1 ）。由于 *H200* 测试平台上的磁盘受带宽限制（ <1 GiB/s ），我们使用 H20 存储 平台上的 DeepSeek-V3 [12] 模型单独评估这一效果。如图 13 所示，在请求速率为 12 req/s 时，页优先布局在已经相当大的页面大小基础上，将平均 TTFT 提升了 2.1× ，并将吞吐量提升了 1.3×。

## 5.4 GH200 机器上的基准测试

最后，我们探讨了 Strata 如何与新兴硬件趋势相契合，这些趋势大幅提升了 CPU 与 GPU 之间的带宽。为此，我们在 H200 和 GH200 平台上对 *SGLang-HiCache* 、 *Strata-IO* 和 *Strata* 进行了基准测试。我们还报告了一个 *Strata-Oracle* 平台，该平台模拟了 CPU 与 GPU 之间具有无限带宽的系统所能达到的 TTFT。

图 14 报告了 *SGLang* 、 *Strata-IO* 和 *Strata* 在这两个平台上所达到的平均 TTFT 基准 -

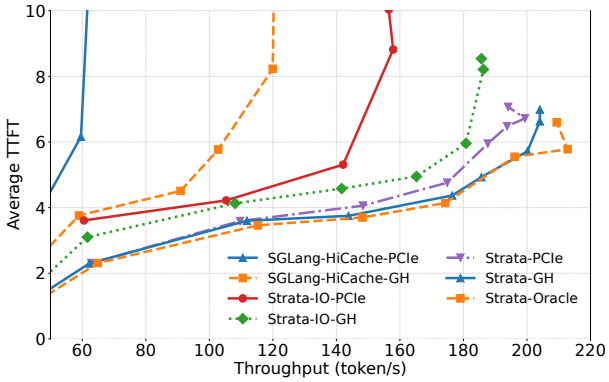



图 14：PCIe-5.0 与 Grace-Hopper 平台基准测试的放大对比。

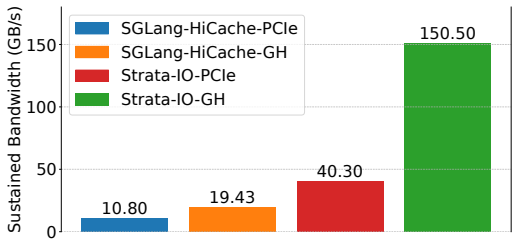



图 15：持续带宽对比。

使用 LooGLE 数据集对 Llama-3.1-8B 模型进行基准测试。图 15 报告了同一任务的平均持续带宽。我们观察到，与我们之前的评估结果一致，标准的 DMA 管理内存拷贝在不使用更大页面的情况下无法有效利用硬件改进。虽然 GH200 改进的带宽降低了 SGLang 基线的延迟，但仅靠硬件改进甚至无法在 H200 平台上超越 Strata-IO。

相比之下，使用 *Strata-IO* 将持续的 host-GPU 传输带宽从 40 GB/s 提升至 150 GB/s。然而，仍需要调度方面的改进才能充分利用 Grace Hopper 等平台。虽然 *Strata-IO-GH* 通过更高带宽实现了更高吞吐量，缓解了 I/O 停顿，但它甚至仍未超越 *Strata-PCIe*。总之，Strata 改进的 I/O 传输机制与其带宽感知调度器能够充分利用 Grace Hopper 等新兴平台，实现与 Oracle 设置相当的性能。这些基准测试表明，虽然互连带宽的增加是有益的，但现有软件往往未能充分利用它。理想情况下，如 *Strata-GH* 接近 Oracle 的性能所展示的那样，利用新的硬件能力可以解锁新的可能性。具体而言，该平台在专业化、高性价比且高性能的长上下文服务方面展现出前景。

## 6 讨论

在展示了 Strata 的优势之后，我们现在讨论其局限性以及未来工作的方向。

**I/O 内核干扰**。正如 §4.2 中所分析，Strata 的 GPU 辅助 I/O 以少量的 SM 争用为代价，在小而碎片化的页面上实现了高传输效率。CUDA 12.8 最近引入了 `cudaMemcpyBatchAsync` [38]，它通过将许多小型复制操作批处理为单次主机驱动提交来提高传输效率。由于它使用 GPU DMA 引擎，因此不会与并发计算争用 SM 资源。在用于图 5 的同一 H200 微基准测试中，其吞吐量低于我们的 GPU 辅助 I/O 内核，为 38 GB/s 对 48 GB/s。虽然这一较低的吞吐量可能会在加载路径上引入额外的停顿，但它使该 API 非常适合不太关键的 GPU 到 CPU 备份路径。因此，Strata 可以配置为对 CPU 到 GPU 加载使用 GPU 辅助 I/O 内核，对 GPU 到 CPU 备份使用批复制 API，从而在实现有竞争力的带宽的同时进一步减少内核干扰。展望未来，我们计划进一步降低 GPU 辅助内核的开销，并相信这些结果将推动更通用的片上 I/O 加速器的发展。

**调度中的公平性**。尽管 §4.3.2 引入了一种防止饥饿的机制，但 Strata 的调度器优先考虑聚合 I/O 和计算效率，仍可能不公平地对待请求，从而使单个请求面临违反服务级别目标（SLO）的风险。一个自然的扩展是在分配资源时使批处理形成具备 SLO 意识和公平性意识。当前策略还侧重于隐藏将数据从主机 DRAM 移动到 HBM 的延迟；同样有前景的方向是为隐藏从较慢存储层预取的延迟创造调度余量，从而超越 Strata 当前将预取与排队延迟重叠的方法。

**模型覆盖范围**。Strata 针对标准注意力机制产生的密集 KV 缓存进行分层缓存。随着稀疏注意力和线性注意力等混合机制逐渐被采用，将同等水平的支持扩展到这些模型变得越来越重要，并在内存管理和调度策略设计方面带来了独特的挑战。

## 7 相关工作

**上下文缓存与共享**。先前的工作已经探索了多种复用 KV 缓存以避免冗余计算的机制。SGLang [53] 使用基数树（RadixTree）来跟踪共享上下文，而其他服务引擎如 vLLM [26] 和 Mooncake [43] 采用哈希方案，从 token ID 和前缀页哈希生成唯一的页标识符。LMDeploy [10] 采用了一种具有更粗粒度 trie 的混合设计。Strata 在 SGLang 的基础上，将其 RadixTree 扩展为 HiRadixTree。除了精确前缀复用之外，CacheGen [29] 和 CacheBlend [50] 探索了近似 KV 缓存共享。

相比之下，Strata 专注于精确缓存，且不影响模型准确率。Marconi [42] 为混合 LLM 增加了前缀缓存支持，这与 Strata 互补，是将我们的框架扩展到混合模型的一个有前景的方向。

**KV 缓存卸载**。近期已有若干工作探索利用二级内存层级（例如 CPU DRAM 、 SSD ）进行 KV 缓存存储，并在需要时加载用于计算。CachedAttention [14] 和 Pensieve [52] 均采用逐层策略，将 KV 缓存加载与计算重叠。FlashGen [21] 通过重排序执行调度进一步增强了这一流水线，该调度已在 SGLang 中实现，并用于我们的基线设置中。

**高效的主机 -GPU I/O**。从业者早已认识到 GPU 可以加速 GPU 与 CPU 内存之间的小数据移动 [33, 44]。Strata 引入了一种专为 LLM 服务定制的 GPU 辅助 KV 缓存传输机制，并且据我们所知，首次对传输效率、缓存命中率和干扰开销之间的权衡进行了定量分析。GDRCopy [24] 和 CUDA 统一内存 [37] 能够实现低延迟的细粒度主机 -GPU 数据传输，但其设计难以维持大规模 KV 缓存传输所需的带宽。

**大规模 KV 缓存解耦**。近期工作提出构建大规模解耦式 KV 缓存内存池和全局资源协调器，以在更大规模上实现缓存收益。Mooncake [43] 利用包括 CPU 、 DRAM 、 SSD 和 NIC 在内的资源来建立解耦式 KV 缓存。MemServe [19] 通过全局调度器和弹性内存池统一了请求间和请求内的 KV 缓存优化。Strata 可以通过与其 KV 缓存传输引擎集成而从这些设计中受益。然而，Strata 专注于单个计算实例内的内存管理和调度，并且本质上不依赖专用硬件（如高速网络）来实现其缓存收益。

## 8 结论

本文提出了 Strata，一种分层上下文缓存框架，用于解决长上下文 LLM 服务的关键瓶颈。通过结合 GPU 辅助 I/O 以缓解 KV 缓存碎片化，以及平衡计算与数据传输的缓存感知调度，Strata 提高了在不同延迟区间下的利用率。我们的评估表明，Strata 在长上下文基准测试中优于最先进的系统，同时在短上下文工作负载上保持强劲性能，从而确立了 Strata 作为高效长上下文 LLM 服务的实用且可扩展解决方案的地位。

## 致谢

我们感谢匿名审稿人提出的建设性意见，这些意见有助于改进本文。我们感谢 Lingfan Yu 就 Pensieve 进行的有益讨论。我们还感谢来自 SGLang 开源社区的 Sicheng Pan 、 Tingwei Huang 、 Zhangheng Huang 、 Shuwen Wang 、 Teng Ma 、 Mingxing Zhang 、 Arnav Balyan 、 Moein Khazraee 、 Zhenwei Pi 、 Yi Zhang 、 Shangming Cai 、 Shiyang Chen 、 Ke Yang 和 Ying Sheng 提供的宝贵反馈和贡献。本研究部分得到了斯坦福平台实验室及其附属机构的支持。Zhiqiang Xie 获得了 NVIDIA 研究生奖学金的支持。我们感谢 NVIDIA 和 Nebius 提供计算资源。

## 参考文献

[1] Artificial analysis 模型排行榜。https://artificialanalysis.ai/leaderboards/models。

[2] Advanced Micro Devices,Inc. ROCm™ 软件 6.4.3 文档。https://rocm.docs.amd.com/en/latest/，2025 年 6 月。[3] Meta AI。推出 llama 3.1：我们迄今为止能力最强的模型。https://ai.meta.com/blog/meta-llama-3-1/，2024 年 7 月。[4] Meta AI。llama 4 系列：原生多模态 AI 创新新时代的开端。https://ai.meta.com/blog/llama-4-multimodal-intelligence/，2025 年 4 月。

[5] Anthropic。提示缓存。https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching，2024 年。

[6] Anthropic。Claude code：智能体编程助手。https://code.claude.com/，2025 年。

[7] Anthropic。Claude sonnet 4 现已支持 100 万 token 的上下文。https://claude.com/blog/1m-context，2025。

[8] Nirav Atre 、 Justine Sherry 、 Weina Wang 和 Daniel S. Berger。带延迟命中的缓存。载于 ACM 数据通信专业组年会论文集：计算机通信应用、技术、架构与协议，SIGCOMM '20，第 495–513 页，美国纽约州纽约市，2020。美国计算机协会。

[9] Qizhe Cai 、 Shubham Chaudhary 、 Midhul Vuppalapati 、 Jaehyun Hwang 和 Rachit Agarwal。理解主机网络栈开销。载于 2021 年 ACM SIGCOMM 2021 会议论文集，第 65–77 页，美国纽约州纽约市，2021。美国计算机协会。

[10] LMDeploy 贡献者。Lmdeploy：一个用于压缩、部署和服务 LLM 的工具包。https://github.com/InternLM/lmdeploy，2023。[11] DeepSeek。提示缓存。https://api-docs.deepseek.com/guides/kv_cache。

[12] DeepSeek-AI。Deepseek-v3 技术报告。https://arxiv.org/abs/2412.19437，2025。

[13] Assaf Eisenman 、 Asaf Cidon 、 Evgenya Pergament 、 Or Haimovich 、 Ryan Stutsman 、 Mohammad Alizadeh 和 Sachin Katti。Flashield：一种控制闪存写入放大的混合键值缓存。载于第 16 届 USENIX 网络系统设计与实现研讨会（NSDI 19），第 65–78 页，2019。

[14] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 面向多轮对话的高效大语言模型服务：基于缓存注意力机制 . 载于 2024 年 USENIX 会议 USENIX 年度技术会议论文集，USENIX ATC'24，美国，2024 年。USENIX 协会。[15] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 面向大语言模型的检索增强生成：综述 . arXiv 预印本 arXiv:2312.10997，2(1)，2023 年。[16] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 提示缓存：面向低延迟推理的模块化注意力复用 . 载于 P. Gibbons 、 G. Pekhimenko 和 C. De Sa 编辑，机器学习与系统会议论文集，第 6 卷，第 325–338 页，2024 年。[17] Google. 提示缓存 . https://cloud.google.com/vertex-ai/generative-ai/docs/context-cache/context-cache-overview.[18] Google DeepMind. Gemini：Google DeepMind 最具能力且最通用的 AI 模型 . https://deepmind.google/technologies/gemini/，2025 年。[19] Cunchen Hu, Heyang Huang, Junhao Hu, Jiang Xu, Xusheng Chen, Tao Xie, Chenxi Wang, Sa Wang, Yun- gang Bao, Ninghui Sun, and Yizhou Shan. Memserve：面向解耦式 LLM 服务的上下文缓存与弹性内存池，2024 年。[20] Changho Hwang, KyoungSoo Park, Ran Shu, Xinyuan Qu, Peng Cheng, and Yongqiang Xiong. ARK：面向分布式深度学习的 GPU 驱动代码执行 . 载于

第 20 届 USENIX 网络系统设计与实现研讨会（NSDI 23），第 87–101 页，马萨诸塞州波士顿，2023 年 4 月。USENIX 协会。

[21] Jinwoo Jeong and Jeongseob Ahn. 通过高效资源管理加速多轮对话的大语言模型服务。载于第 30 届 ACM 国际编程语言与操作系统架构支持会议论文集，第 2 卷，第 1–15 页，美国纽约州纽约市，2025 年。美国计算机协会。

[22] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Shufan Liu, Xuanzhe Liu, and Xin Jin. Ragcache：面向检索增强生成的高效知识缓存。ACM 计算机系统汇刊，44(1)，2025 年 11 月。

[23] Shuowei Jin, Xueshen Liu, Qingzhao Zhang, and Zhuoqing Mao. 计算还是卸载 KV 缓存？为何不两者兼用？载于国际机器学习会议，第 28031–28043 页。PMLR，2025 年。

[24] Kawthar Shafei Khorassani, Ching-Hsiang Chu, Hari Subramoni, and Dhabaleswar K. Panda. 在支持 GPU 的 OpenPOWER 架构上 MPI 库的性能评估：早期经验。载于高性能计算：ISC 高性能 2019 国际研讨会，德国法兰克福，2019 年 6 月 16-20 日，修订精选论文，第 361–378 页，柏林，海德堡，2019 年。Springer-Verlag。

[25] Tomá Koˇciský, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, Gábor Melis, and Edward Grefenstette. NarrativeQA 阅读理解挑战。计算语言学协会汇刊，6:317–328，2018 年。

[26] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 面向大语言模型服务的高效内存管理与分页注意力机制 . 载于第 29 届操作系统原理研讨会论文集，第 611–626 页，美国纽约州纽约市，2023 年。美国计算机协会。

[27] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 面向知识密集型自然语言处理任务的检索增强生成 . 载于第 34 届国际神经信息处理系统会议论文集，美国纽约州红钩，2020 年。Curran Associates Inc.

[28] Jiaqi Li, Mengmeng Wang, Zilong Zheng, and Muhan Zhang. LooGLE：长上下文语言模型能否理解长上下文？载于第 62 届计算语言学协会年会论文集（第 1 卷：长论文），第 16304–16333 页，泰国曼谷，2024 年。计算语言学协会。

[29] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, 等 . Cachegen：面向快速大语言模型服务的 KV 缓存压缩与流式传输 . 载于 ACM SIGCOMM 2024 会议论文集，第 38–56 页，2024 年。

[30] LMCache. Lmcache. https://lmcache.ai，2025 年。

[31] Lingxiao Ma, Zhiqiang Xie, Zhi Yang, Jilong Xue, Youshan Miao, Wei Cui, Wenxiang Hu, Fan Yang, Lintao Zhang, and Lidong Zhou. Rammer：利用 rTasks 实现整体深度学习编译器优化。载于第 14 届 USENIX 操作系统设计与实现研讨会（OSDI 20），第 881–897 页。USENIX 协会，2020 年 11 月。

[32] Sara McAllister, Benjamin Berg, Julian Tutuncu-Macias, Juncheng Yang, Sathya Gunasekar, Jimmy Lu, Daniel S. Berger, Nathan Beckmann, and Gregory R. Ganger. Kangaroo：在 ASH 上缓存数十亿微小对象。载于 ACM SIGOPS 第 28 届操作系统原理研讨会论文集，第 243–262 页，美国纽约州纽约市，2021 年。美国计算机协会。

[33] Seung Won Min, Vikram Sharma Mailthody, Zaid Qureshi, Jinjun Xiong, Eiman Ebrahimi, and Wen-mei Hwu. Emogi：GPU 中内存外图遍历的高效内存访问。*Proc. VLDB Endow.*，14(2):114–127，2020 年 10 月。

[34] NVIDIA。借助 NVIDIA TensorRT-LLM KV 缓存早期复用，首令牌时间缩短至五分之一。https://developer.nvidia.com/blog/5x-faster-time-to-first-token-with-nvidia-tensorrt-llm-kv-cache-early-reuse/，2024 年。

[35] NVIDIA。NVIDIA GH200 Grace Hopper 超级芯片。https://www.nvidia.com/en-us/data-center/grace-hopper-superchip/，2025 年。

[36] NVIDIA。NVIDIA TensorRT-LLM。https://docs.nvidia.com/tensorrt-llm/index.html，2025 年。

[37] NVIDIA 公司。4.1. 统一内存。https://docs.nvidia.com/cuda/archive/13.1.0/，2025 年。

[38] NVIDIA 公司。CUDA 运行时 API：内存管理。https://docs.nvidia.com/cuda/archive/12.8.1/cuda-runtime-api/group__CUDART__MEMORY.html，2025 年。

[39] NVIDIA 公司。并行线程执行。https://docs.nvidia.com/cuda/parallel-thread-execution/，2025。

[40] OpenAI。ChatGPT 的记忆功能与新增控制项。https://openai.com/index/memory-and-new-controls-for-chatgpt/，2024。

[41] OpenAI。提示缓存。https://platform.openai.com/docs/guides/prompt-caching，2024。

[42] Rui Pan、Zhuang Wang、Zhen Jia、Can Karakus、Luca Zancato、Tri Dao、Yida Wang 和 Ravi Netravali。Marconi：面向混合 LLM 时代的预填充缓存。机器学习与系统会议论文集，7，2025。

[43] Ruoyu Qin、Zheming Li、Weiran He、Jialei Cui、Feng Ren、Mingxing Zhang、Yongwei Wu、Weimin Zheng 和 Xinran Xu。Mooncake：以更多存储换取更少计算 —— 一种面向 LLM 聊天机器人服务的以 KVCache 为中心的架构。载于第 23 届 USENIX 文件与存储技术会议（FAST 25），第 155–170 页，加利福尼亚州圣克拉拉，2025 年 2 月。USENIX 协会。

[44] Zaid Qureshi、Vikram Sharma Mailthody、Isaac Gelado、Seungwon Min、Amna Masood、Jeongmin Park、Jinjun Xiong、C. J. Newburn、Dmitri Vainbrand、I-Hsin Chung、Michael Garland、William Dally 和 Wen-mei Hwu。BAM 系统架构中由 GPU 发起的按需高吞吐存储访问。载于第 28 届 ACM 编程语言与操作系统架构支持国际会议论文集，第 2 卷，第 325–339 页。ACM，2023 年 1 月。

[45] Cheng Tan, Dongxin Lyu, Siyuan Li, Zhangyang Gao, Jingxuan Wei, Siqi Ma, Zicheng Liu, and Stan Z Li. 同行评审作为具有基于角色交互的多轮长上下文对话 . arXiv 预印本 arXiv:2406.05688, 2024.

[46] Qwen 团队 . Qwen2.5-1m：部署您自己的上下文长度高达 1m token 的 qwen. https://qwenlm.github.io/blog/qwen2.5-1m/，2025 年 1 月 .

[47] SGLang 团队 . Sglang v0.4：零开销批调度器、缓存感知负载均衡器、更快的结构化输出 . https://lmsys.org/blog/2024-12-04-sglang-v0-4/, 2024 年 12 月 .

[48] vLLM 贡献者 . vllm 配置 API 参考 . https://docs.vllm.ai/en/latest/api/vllm/vllm.config.html, 2025.

[49] Minzheng Wang, Longze Chen, Fu Cheng, Shengyi Liao, Xinghua Zhang, Bingli Wu, Haiyang Yu, Nan Xu, Lei Zhang, Run Luo, Yunshui Li, Min Yang, Fei Huang, and Yongbin Li. 不遗漏任何文档：使用扩展多文档问答对长上下文 LLM 进行基准测试 . 载于 Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen 编 ,2024 年自然语言处理实证方法会议论文集 , 第 5627–5646 页 , 美国佛罗里达州迈阿密 , 2024 年 11 月 . 计算语言学协会 .

[50] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In Proceedings of the Twentieth European Conference on Computer Systems, EuroSys '25, page 94–109, New York, NY, USA, 2025. Association for Computing Machinery.

[51] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIXSymposium on Operating SystemsDesignandImplementation (OSDI 22)*, pages 521–538, Carlsbad, CA, July 2022. USENIX Association.

[52] Lingfan Yu, Jinkun Lin, and Jinyang Li. Stateful large language model serving with pensieve. In *Proceedings of the Twentieth European Conference on Computer Systems*, EuroSys '25, page 144–158, New York, NY, USA, 2025. Association for Computing Machinery.

[53] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: Ef cient execution of structured language model programs. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advancesin Neural InformationProcessing Systems*, volume 37, pages 62557–62583. Curran Associates, Inc., 2024.

[54] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Dist-serve：为优化有效吞吐量的大语言模型服务解耦预填充与解码。载于第 18 届 USENIX 操作系统设计与实现会议论文集，OSDI'24，美国，2024 年。USENIX 协会。

[55] Kan Zhu, Yufei Gao, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Tian Tang, Qinyu Xu, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, Ziren Wang, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. Nano flow：迈向最优的大语言模型服务吞吐量。载于第 19 届 USENIX 操作系统设计与实现会议论文集，OSDI '25，美国，2025 年。USENIX 协会。